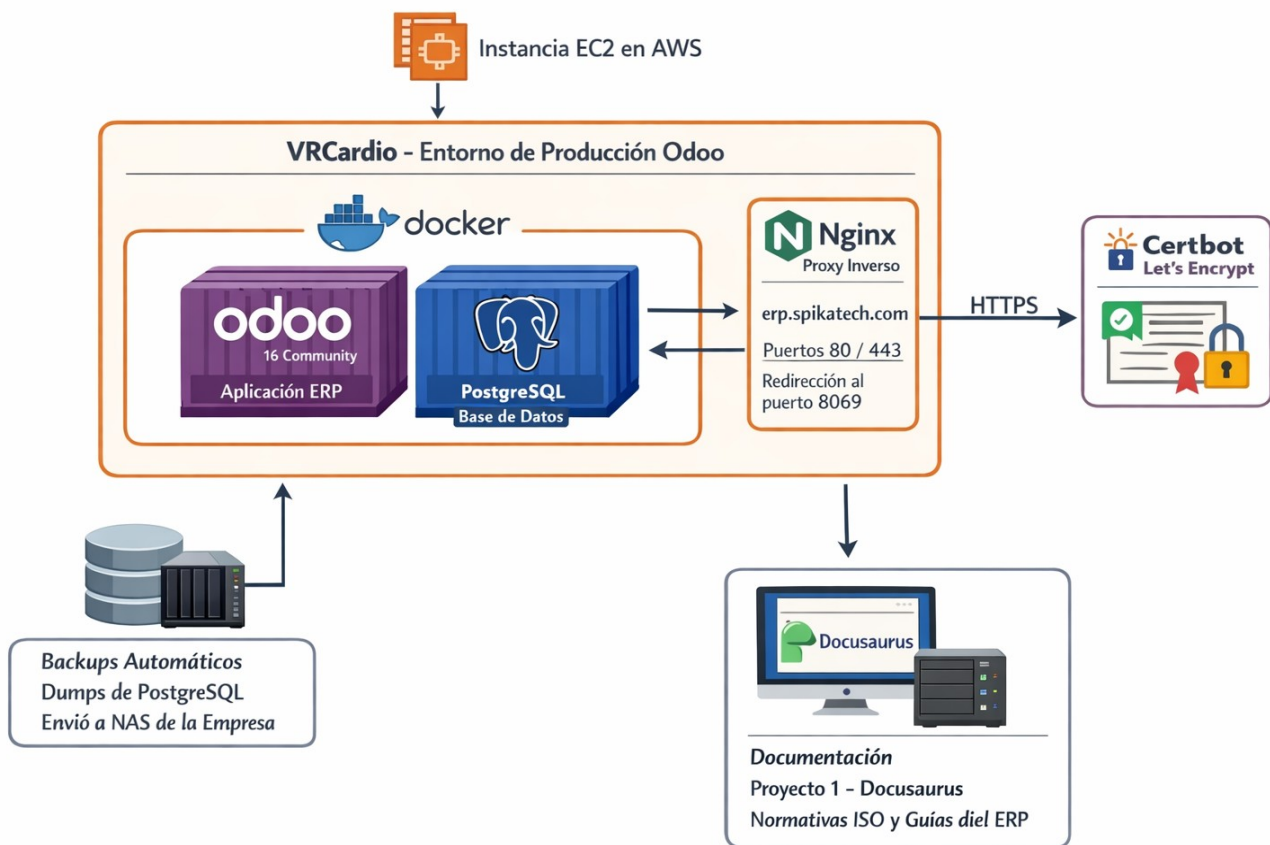


Informe final de prácticas

Proyecto 2 – Implementación y centralización de sistemas de gestión ERP/CRM



Damaris Antonela Antón Oltean

Introducción

En este proyecto he montado un ERP para VRCardio / Spika Tech con Odoo Community 16, empezando desde un entorno de pruebas en local hasta dejarlo desplegado en la nube. Primero probé Odoo con Docker y lo comparé con Dolibarr y ERPNext para decidir qué sistema encajaba mejor.

Una vez elegido Odoo, configuré la infraestructura sobre una instancia de AWS con Odoo y PostgreSQL en contenedores Docker, Nginx como proxy inverso con HTTPS y un sistema de copias de seguridad automáticas. Sobre ese Odoo de producción he ido activando y ajustando los módulos de Ventas, CRM, Proyectos, RRHH e Inventario, definiendo el pipeline comercial (Lead → Contacto → Demo → Cierre), modelando las licencias como productos de servicio y creando las primeras integraciones con scripts en Python usando la API XML-RPC.

Semana 1 - Levantamiento de Requisitos y Odoo Base

Introducción

Objetivo de la semana: Configurar un entorno de pruebas con Odoo Community Edition 16 y evaluar los módulos principales para cubrir los procesos de ventas, facturación, inventario y gestión de proyectos de VRCardio.

Requisitos básicos de VRCardio para el ERP

En el menú **Aplicaciones** he instalado los módulos que necesito para cubrir los requisitos de la práctica. Estos son los que he activado y para qué los voy a usar:

- **Ventas:** para crear presupuestos y pedidos de venta de las licencias de VRCardio y servicios relacionados.
- **Facturación:** para generar y validar las facturas a partir de los pedidos de venta.
- **CRM:** para llevar el seguimiento de leads y oportunidades (hospitales interesados, demos, etc.).
- **Inventario:** para definir los productos y servicios, en este caso las licencias de VRCardio (como productos de tipo servicio).
- **Proyecto:** para crear proyectos de implantación y desarrollo, con sus tareas y responsables.
- **Empleados:** para gestionar la información básica de los empleados y preparar la parte de RRHH.
- **Contactos:** para dar de alta hospitales y personas de contacto.
- **Calendario:** para planificar reuniones, demos y eventos.
- **Sitio web:** como base para una posible web/portal más adelante.
- **Conversaciones (chat):** para comunicación interna básica entre usuarios dentro de Odoo.

Despliegue de Odoo 16 Community en sandbox

Para el entorno de pruebas he montado Odoo Community 16 con Docker, junto a una base de datos PostgreSQL. Todo el entorno se encuentra en la carpeta `semana1_odoo_sandbox` dentro del proyecto.

1. Preparar la carpeta del proyecto y subcarpetas

```
mkdir -p proyecto-erp
cd proyecto-erp
mkdir -p semana1_odoo_sandbox/{addons,config,odoo-db-data}
```

2. Crear fichero docker-compose.yml dentro de la carpeta `semana1odoosandbox`

```
nano docker-compose.yml
```

```

services:
  db:
    image: postgres:15
    restart: always
    environment:
      - POSTGRES_DB=postgres
      - POSTGRES_USER=odoo
      - POSTGRES_PASSWORD=odoo123
    volumes:
      - ./odoo-db-data:/var/lib/postgresql/data # Guarda la BBDD en esta carpeta

  odoo:
    image: odoo:16
    restart: always
    depends_on:
      - db
    ports:
      - "8069:8069"
    environment:
      - DB_HOST=db
      - DB_USER=odoo
      - DB_PASSWORD=odoo123
    volumes:
      - ./addons:/mnt/extra-addons # Conecta la carpeta addons
      - ./config:/etc/odoo # Conecta la carpeta config

```

Nota: las credenciales son simples ya que se trata de un entorno de pruebas (sandbox). En producción se usarían variables de entorno seguras.

3. Crear fichero odoo.conf

En la carpeta /config

```
nano odoo.conf
```

```
[options]
  addons_path = /mnt/extra-addons
  data_dir = /var/lib/odoo
  db_host = db
  db_user = odoo
  db_password = odoo123
  db_port = 5432
```

Nota: Este fichero permite externalizar la configuración del servidor Odoo y facilita futuras modificaciones sin reconstruir la imagen Docker.

4. Iniciar el despliegue

```
> docker compose up -d          # levanta Odoo y PostgreSQL en segundo plano
> docker ps                     # comprobar que los contenedores están corriendo
> docker compose down           # si se quiere parar el entorno
```

5. Crear base de datos

Desde el navegador (<http://localhost:8069>), he creado la base de datos vrcardio_test con el usuario administrador.

Pruebas Odoo

Ejemplo 1: Cliente (hospital) en Contactos

Ruta: Contactos → Nuevo

- **Nombre:** Hospital Demo VRCardio
- Compañía
- **Dirección:** C/ Salud 123, 28001 Madrid, España
- **NIF:** B12345678
- **Teléfono:** 910 000 111
- **Móvil:** 600 000 222
- **Correo electrónico:** contacto@hospitaldemo.com
- **Sitio web:** <https://www.hospitaldemo.com>
- **Etiquetas:** Hospital, Cliente VRCardio

Ejemplo 2: Producto Licencia VRCardio Anual

Ruta: Inventario → Productos → Nuevo

- **Nombre del producto:** Licencia VRCardio Anual
- Puede ser vendido
- **Tipo de producto:** Servicio
- **Precio de venta:** 5000
- **Impuestos del cliente:** IVA 21 %
- **Categoría de producto:** Licencias VR
- **Referencia interna:** LIC-VRC-ANUAL
- **Notas internas:** Licencia anual de VRCardio para un hospital

Ejemplo 3: Presupuesto de venta

Ruta: Ventas → Pedidos → Presupuesto de venta → Nuevo

- **Cliente:** Hospital Demo VRCardio
- **Línea de pedido:**
 - Producto: Licencia VRCardio Anual
 - Descripción: Licencia anual VRCardio
 - Cantidad: 1
 - Precio unitario: 5000 (se rellena solo)

Ejemplo 4: Proyecto de implantación

Ruta: Proyecto → Nuevo

- **Nombre del proyecto:** Implantación VRCardio - Hospital Demo

A continuación, dentro del proyecto, se crea la tarea:

- **Nombre de la tarea:** Instalación y configuración inicial
- **Personas asignadas:** Administrator
- **Cliente:** Hospital Demo VRCardio
- **Fecha límite:** 28/02/2026

Ejemplo 5: Empleado de prueba

Ruta: Empleados → Nuevo

- **Nombre:** Técnico VR
 - **Puesto:** Técnico de implantación
 - **Correo de trabajo:** tecnico@vrcardio.com
-

Conclusiones

Durante la primera semana se ha conseguido:

1. Identificar los módulos esenciales de Odoo para VRCardio.
 2. Montar un entorno de pruebas funcional con Docker y PostgreSQL.
 3. Realizar pruebas iniciales de creación de clientes, productos, presupuestos, proyectos y empleados.
 4. Próximo paso: comparar los flujos de Odoo con otros ERP.
-

Semana 2 - Despliegue de Alternativas

Introducción

Objetivo de la semana: Desplegar dos alternativas Open Source a Odoo (Dolibarr y ERPNext) en modo sandbox usando Docker Compose, y preparar una prueba de concepto replicando un flujo clave de Spika Tech.

Selección de alternativas Open Source

Para comparar Odoo con otras soluciones ERP libres, he seleccionado:

- **Dolibarr ERP & CRM:** ERP ligero, modular y orientado a pymes, con módulos de ventas, facturación, proyectos y otros similares a los de Odoo.
- **ERPNext:** ERP más completo, con módulos de contabilidad, proyectos, CRM, soporte y otros procesos empresariales avanzados.

Ambas herramientas cuentan con imágenes oficiales o repositorios preparados para Docker, lo que facilita su despliegue rápido en un entorno de pruebas.

Despliegue de Dolibarr en sandbox

Para comparar con Odoo he montado un entorno de pruebas de Dolibarr usando Docker y MariaDB. Todo está dentro de la carpeta `semana2_dolibarr_sandbox` del proyecto.

1. Preparar la carpeta del proyecto y subcarpetas

```
cd ~/proyecto-erp
mkdir -p semana2_dolibarr_sandbox/{dolibarr-db,dolibarr-docs}
```

Nota: separo el volumen de base de datos (dolibarr-db) y el de documentos (dolibarr-docs) para poder conservar los datos aunque borre los contenedores y reutilizar el sandbox en otras semanas.

2. Crear fichero docker-compose.yml dentro de la carpeta semana2dolibarrsandbox

```
nano docker-compose.yml
```

```
services:
  mariadb:
    image: mariadb:10.11
    restart: always
    environment:
      - MYSQL_ROOT_PASSWORD=root
      - MYSQL_DATABASE=dolibarr
      - MYSQL_USER=dolibarr
      - MYSQL_PASSWORD=dolibarr123
    volumes:
      - ./dolibarr-db:/var/lib/mysql

  dolibarr:
    image: dolibarr/dolibarr:latest
    restart: always
    depends_on:
      - mariadb
    ports:
      - "8081:80"
    environment:
      - DOLI_DB_HOST=mariadb
      - DOLI_DB_NAME=dolibarr
      - DOLI_DB_USER=dolibarr
      - DOLI_DB_PASSWORD=dolibarr123
    volumes:
      - ./dolibarr-docs:/var/www/documents
```

3. Iniciar el despliegue

```
> docker compose up -d          # levanta Odoo y PostgreSQL en segundo plano
> docker ps                     # comprobar que los contenedores están corriendo
> docker compose down           # si se quiere parar el entorno
```


Desde el navegador (<http://localhost:8081>), se puede entrar con el login admin y contraseña admin (valores por defecto de la imagen oficial).

Despliegue de ERPNext en sandbox

Para ERPNext he elegido un despliegue sencillo usando una imagen Docker “all-in-one” (incluye MariaDB, Redis, Frappe y Nginx en un solo contenedor), suficiente para pruebas y comparativas con Odoo y Dolibarr. Todo está dentro de la carpeta `semana2_erpnext_sandbox` del proyecto.

1. Preparar la carpeta del proyecto y subcarpetas

```
cd ~/proyecto-erp
mkdir -p semana2_erpnext_sandbox
```

2. Crear fichero docker-compose.yml dentro de la carpeta `semana2erpnextsandbox`

```
nano docker-compose.yml
```

```
services:
  erpnext:
    image: zhjh685/erpnext:15-all-in-one
    container_name: erpnext15
    restart: always
    ports:
      - "8080:80"
    environment:
      DB_ROOT_PASSWORD: root
      SITE_NAME: vrcardio.local
      ADMIN_PASSWORD: admin123
      INSTALL_APPS: erpnext
    volumes:
      - erpnext_sites:/home/frappe/frappe-bench/sites

volumes:
  erpnext_sites:
```

3. Iniciar el despliegue

```
> docker compose up -d          # levanta Odoo y PostgreSQL en segundo plano
> docker ps                    # comprobar que los contenedores están corriendo
> docker compose down          # si se quiere parar el entorno
```

4. Problemas encontrados

Durante el despliegue de ERPNext aparecieron varios problemas relacionados con permisos y configuración inicial del sitio:

- **Errores de permisos con el usuario Administrator:** después del asistente inicial, al intentar acceder a distintas pantallas con el usuario `Administrator` el sistema devolvía mensajes del tipo *“Method Not Allowed / You are not permitted to access this resource”*. Esto impedía continuar la configuración desde la interfaz web.
- **Idioma del sistema incorrecto:** al configurar el idioma en el asistente, la interfaz cambiaba a chino en lugar de aplicar el español. Para forzar el idioma global del sitio a español ejecuté en el contenedor:

```
> docker exec -it erpnext15 bash
> bench --site www.gdjoian.com set-config language es
```

- **Creación de un usuario administrador alternativo vía consola:** para recuperar el acceso de administración, creé un nuevo usuario con permisos de System Manager usando la consola de Frappe.

```
> docker exec -it erpnext15 bash
> bench --site www.gdjoian.com console
```

```

from frappe.utils.password import update_password
import frappe

# crea un usuario nuevo si no existe
if not frappe.db.exists("User", "admin@vrcardio.local"):
    user = frappe.get_doc({
        "doctype": "User",
        "email": "admin@vrcardio.local",
        "first_name": "Admin VRCardio",
        "enabled": 1,
        "language": "es",
        "time_zone": "Europe/Madrid"
    })
    user.insert(ignore_permissions=True)
else:
    user = frappe.get_doc("User", "admin@vrcardio.local")

# asignar rol de administrador (System Manager)
user.add_roles("System Manager")
user.save(ignore_permissions=True)

# establecer contraseña
update_password("admin@vrcardio.local", "clave$1")
frappe.db.commit()
exit()

```

Desde el navegador (<http://localhost:8080>), el acceso final se hace con el correo admin@vrcardio.local y contraseña clave\$1.

Pruebas Dolibarr

Activación de módulos en Dolibarr

Antes de realizar las pruebas he habilitado en Dolibarr los módulos necesarios para replicar el flujo definido en Odoo:

- **Terceros:** permite gestionar empresas y contactos (clientes y clientes potenciales), equivalente a los contactos/clientes de Odoo.
- **Presupuestos:** añade el módulo de propuestas comerciales, que usaré para crear el presupuesto de la licencia VRCardio anual.
- **Productos:** habilita la gestión de productos, imprescindible para definir el catálogo de licencias.

- **Servicios:** complementa al módulo de productos para poder crear la “Licencia VRCardio Anual” como servicio en lugar de producto físico.
- **Proyectos u Oportunidades:** permite crear proyectos y tareas asociadas al cliente, que utilizaré para documentar la implantación en el hospital.

Con estos módulos activos, se facilitará la comparación de los flujos de trabajo.

Configuración de la empresa

Ruta: Inicio → Configuración → Empresa/Organización → Empresa

- **Razón social:** Spika Tech
- **Divisa principal:** EUR
- **País:** España
- **Dirección:** C/ Ejemplo 1, 28001 Madrid, España
- **Capital:** 3000
- **Tipo de entidad comercial:** Sociedad limitada (o equivalente)
- **Objeto de la empresa:** Desarrollo de soluciones VR para salud

En la sección fiscal:

- **Mes de inicio de ejercicio:** Enero
- **Gestión de IVA:** “Sujeto a IVA”

Nota: con esta configuración, Dolibarr permite asignar tipos de IVA (por ejemplo, 21 %) a los servicios como “Licencia VRCardio Anual” y usar correctamente los impuestos en presupuestos y facturas.

Ejemplo 1: Cliente (hospital) como Tercero

Ruta: Terceros → Nuevo tercero → Cliente

- **Nombre:** Hospital Demo VRCardio
- **Tipo:** Cliente
- **Dirección:** C/ Salud 123, 28001 Madrid, España
- **Teléfono:** 910 000 111
- **Móvil:** 600 000 222
- **Sitio web:** <https://www.hospitaldemo.com>
- **Correo electrónico:** contacto@hospitaldemo.com
- **NIF/CIF:** B12345678
- **Tipo de tercero:** Gran empresa

Ejemplo 2: Producto Licencia VRCardio Anual

Ruta: Productos/Servicios → Nuevo servicio

- **Ref. producto:** LIC-VRC-ANUAL

- **Etiqueta:** Licencia VRCardio Anual
- **Descripción:** Licencia anual de VRCardio para un hospital
- **Precio de venta:** 5000
- **Tasa IVA:** 21 %

Ejemplo 3: Proyecto de implantación

Ruta: Proyectos → Nueva oportunidad o proyecto

- **Ref.:** PROJ-VRC-001
- **Etiqueta:** Implantación VRCardio - Hospital Demo
- **Uso:** SIga las tareas o el tiempo dedicado
- **Tercero:** Hospital Demo VRCardio
- **Estado:** Activo
- **Presupuesto:** 5000
- **Fecha:** 06/02/2026 a 28/02/2026
- **Descripción:** Proyecto de implantación de VRCardio en Hospital Demo
- **Visibilidad:** Contactos asignados
- **Asignarme como contacto con el tipo:** Jefe de proyecto

Ejemplo 4: Presupuesto de venta (Propuesta comercial)

Ruta: Comercial → Presupuestos → Nuevo presupuesto

- **Cliente:** Hospital Demo VRCardio
- **Fecha presupuesto:** 06/02/2026
- **Duración de validez:** 30
- **Condiciones de pago:** 30 días
- **Forma de pago:** Transferencia bancaria
- **Proyecto:** PROJ-VRC-001
- **Nota (pública):** Presupuesto de licencia anual VRCardio para Hospital Demo VRCardio
- **Añadir nueva línea:**
 - Productos/servicios predefinidos: LIC-VRC-ANUAL
 - Taxes: 21 %
 - P.U.: 5000
 - Cant.: 1

Ejemplo 5: Empleado de prueba

Ruta: Configuración → Usuarios y grupos → Nuevo usuario

- **Apellido:** Técnico
- **Login:** tecnico.vr
- **Administrador de sistema:** No

- **Supervisor:** SuperAdmin
 - **Contraseña:** AXuaCybHJWDP (por defecto)
 - **Correo:** tecnico@vrcardio.com
 - **Puesto de trabajo:** Técnico VR
-

Pruebas ERPNext

Ejemplo 1: Cliente (hospital)

Ruta: Ventas → Cliente → Crea tu primer cliente

- **Nombre del cliente:** Hospital Demo VRCardio
- **Tipo de Cliente:** Compañía
- **ID de Correo Electrónico:** contacto@hospitaldemo.com
- **Número de teléfono móvil:** 600 000 222
- **Dirección línea 1:** C/ Salud 123
- **Ciudad:** Madrid
- **Estado:** Madrid
- **Código postal:** 28001
- **País:** Spain

Ejemplo 2: Producto Licencia VRCardio Anual

Ruta: Almacén → Producto → Crea tu primer producto

- **Código del Producto:** LIC-VRC-ANUAL
- **Nombre del artículo:** Licencia VRCardio Anual
- **Grupo de Productos:** Servicios
- **Unidad de Medida:** Unidad
- **Precio de venta estándar:** 5000
- **Descripcion:** Licencia anual de VRCardio para un hospital

Ejemplo 3: Presupuesto / Sales Order

Ruta: Ventas → Cotización → Crea tu primer cotización

- **Cliente:** Hospital Demo VRCardio
- **Fecha:** 06/02/2026
- **Válida hasta:** 28/02/2026
- **Tipo de orden:** Ventas

En la tabla **Productos:**

- **Código del Producto:** LIC-VRC-ANUAL: Licencia anual VRCardio

- **Cantidad:** 1
- **Precio:** 5000 (se rellena automáticamente a partir del producto)

Ejemplo 4: Proyecto de implantación

Ruta: Proyecto → Proyecto → Crear: Proyecto

- **Nombre del Proyecto:** Implantación VRCardio - Hospital Demo
- **Fecha prevista de inicio:** 06/02/2026
- **Fecha prevista de finalización:** 28/02/2026
- **Cliente:** Hospital Demo VRCardio

Dentro del proyecto, crear una tarea:

Ruta: Proyectos → Tarea → +

- **Asunto:** Instalación y configuración inicial
- **Proyecto:** PROJ-0001

Ejemplo 5: Empleado de prueba

Ruta: Usuarios → Usuario → Agregar usuario

- **Primer Nombre:** Técnico VR
- **Género:** Masculino
- **Fecha de ingreso:** 06/02/2026
- **Fecha de nacimiento:** 16/08/2002
- **Compañía:** Spika Tech

Conclusiones (Dolibarr y ERPNext)

Durante la segunda semana se ha conseguido:

1. Replicar en Dolibarr el flujo básico de VRCardio: creación de cliente, servicio (licencia), presupuesto, proyecto asociado y usuario técnico con permisos limitados.
 2. Replicar el mismo flujo en ERPNext: cliente, ítem de servicio, cotización, proyecto con tareas y empleado/usuario “Técnico VR”.
 3. Detectar diferencias de usabilidad y permisos entre ambos ERP (creación de productos, gestión de proyectos y configuración de roles/RRHH).
 4. Próximo paso: documentar comparativamente los flujos de venta e implantación en ambos sistemas y evaluar cuál encaja mejor con las necesidades de VRCardio.
-

Semana 3 - Comparativa Técnica y Funcional

Introducción

Objetivo de la semana: Comparar de forma estructurada Odoo, Dolibarr y ERPNext a partir de los flujos que ya he probado en las semanas 1 y 2, y decidir qué ERP encaja mejor con las necesidades de VRCardio y Spika Tech. Para ello se ha elaborado una matriz de decisión con varios criterios (usabilidad, mantenimiento, API y consumo de recursos en el VPS) y un informe de recomendación técnica, evaluando facilidad de uso, coste de mantenimiento, flexibilidad de la API y consumo de recursos en el VPS.

Matriz de decisión: Odoo vs Dolibarr vs ERPNext

A partir de los despliegues y pruebas de las semanas anteriores se ha preparado una matriz de decisión con los criterios que pide el proyecto. La puntuación es de 1 a 5 (5 = mejor valoración), acompañada de una breve justificación basada en la experiencia práctica en los flujos de cliente-licencia-proyecto.

Criterio	Odoo	Dolibarr	ERPNext
Facilidad de uso	4 – Interfaz moderna e intuitiva.	3 – Sencillo y directo, algo básico.	3 – Muy completo, algo abrumador.
Coste de mantenimiento	3 – Algo de trabajo en módulos y updates.	4 – Fácil de mantener y actualizar.	3 – Más complejo por su arquitectura.
Flexibilidad de la API	5 – Muy buena para integraciones.	3 – API limitada, casos simples.	4 – API REST completa, más configuración.
Consumo de recursos en el VPS	3 – Requiere servidor decente, pero manejable.	4 – Muy ligero, ideal para VPS pequeños.	2 – Pesado y exigente en recursos.
Encaje con VRCardio (flujos)	5 – Cubre bien clientes, proyectos y licencias.	4 – Encaja en flujos básicos.	4 – Potente, quizá más de lo necesario.
Uso en móvil	4 – Buena app y versión responsive.	3 – Móvil funcional pero básico.	4 – Experiencia móvil correcta.

Nota: las puntuaciones se basan en los escenarios configurados en las semanas 1 y 2 (cliente hospital, licencia VRCardio anual, presupuesto/cotización y proyecto de implantación).

Análisis comparativo

Odoo

- **Ventajas:**
 - Integración entre módulos (Ventas, CRM, Inventario, Proyectos, RRHH).
 - API madura y flexible para futuras integraciones con el software VRCardio.
 - Buena experiencia móvil (app y versión web responsive).
- **Desventajas:**
 - Requiere un VPS con recursos decentes.
 - Actualizaciones de módulos y mantenimiento algo más complejos.

Dolibarr

- **Ventajas:**
 - Ligero y fácil de mantener en un VPS.
 - Permite el flujo básico de clientes, proyectos y licencias VRCardio.
- **Desventajas:**
 - Interfaz menos homogénea y menos pulida.
 - Funcionalidad más limitada para flujos complejos.

ERPNext

- **Ventajas:**
 - Muy completo y potente a nivel de proyectos y ventas.
 - API REST lista para integraciones externas.
- **Desventajas:**
 - Complejidad inicial alta y curva de aprendizaje mayor.
 - Problemas de permisos e idioma en la configuración inicial.

Recomendación técnica

Teniendo en cuenta los criterios definidos en la matriz de decisión (facilidad de uso, coste de mantenimiento, flexibilidad de la API y consumo de recursos en el VPS), el sistema que mejor se ajusta al proyecto es **Odoo Community 16**.

Por qué Odoo es el elegido

- Ofrece una integración muy buena entre módulos clave (Ventas, CRM, Inventario, Proyectos y RRHH).

- Dispone de una API madura y flexible, ideal para integrarla con el software VRCardio en el futuro.
- Cubre de forma sólida los flujos necesarios: gestión de clientes (hospitales), licencias de software y proyectos de implantación.
- Mantiene un equilibrio razonable entre facilidad de uso y consumo de recursos en el VPS.

Para una empresa como VRCardio, que necesita gestionar licencias de software, proyectos en hospitales y relaciones comerciales, **Odoo ofrece una base sólida y escalable**, centralizando los procesos y facilitando la colaboración entre equipos.

Conclusiones

Durante la tercera semana se ha conseguido:

1. Elaborar una matriz de decisión comparando Odoo, Dolibarr y ERPNext según los criterios solicitados (facilidad de uso, coste de mantenimiento, flexibilidad de la API y consumo de recursos en el VPS).
 2. Analizar las ventajas e inconvenientes de cada ERP a partir de los flujos ya configurados en semanas anteriores, para justificar la elección del sistema final.
 3. Seleccionar Odoo Community 16 como sistema ganador para las siguientes fases del proyecto (instalación en producción, parametrización y migración de datos).
 4. Próximo paso: realizar una instalación limpia de Odoo en el VPS de producción, configurando PostgreSQL de forma segura y añadiendo HTTPS con Let's Encrypt y restricciones de acceso.
-

Semana 4 - Infraestructura de Producción

Introducción

Objetivo de la semana: Realizar una instalación limpia de **Odoo Community 16** como sistema ERP ganador en un entorno de producción, con PostgreSQL configurado de forma más robusta y con medidas de seguridad básicas (HTTPS y restricciones de acceso), preparándolo para su ejecución definitiva en el VPS. En vez de ir directamente al VPS, esta semana he montado primero un entorno de “producción local” en mi portátil usando Docker, como paso intermedio antes del despliegue final.

Instalación limpia de Odoo Community 16

Arquitectura de la infraestructura de producción

La idea es tener cada parte en su sitio: un contenedor para Odoo, otro para PostgreSQL y carpetas en el host donde se guarda toda la información importante.

Componentes:

- **Odoo (aplicación):**

- Imagen `odoo:16` en el contenedor `odoo-prod-app`.
- Lee su configuración desde un fichero `odoo.conf` montado en `/etc/odoo`.

- **PostgreSQL (base de datos):**

- Imagen `postgres:15` en el contenedor `odoo-prod-db`.
- Usuario y contraseña específicos para Odoo (`odoo_prod` / `odoo_prod_pass`).

- **Carpetas del proyecto (volúmenes):**

- `odoo-config` → configuración de Odoo.
- `odoo-addons` → addons extra (de momento vacío).
- `odoo-data` → datos internos de Odoo (sesiones, archivos, etc.).
- `postgres-data` → datos de la base de datos.

En local entro a Odoo por `http://localhost:8069`. Más adelante, en el VPS, se pondrá Nginx delante para entrar por HTTPS con un nombre de dominio.

Navegador → Nginx (en el VPS) → Odoo (contenedor, puerto 8069) → PostgreSQL (contenedor, puerto 5432)

Para no romper nada en producción real, primero he montado este entorno de producción local en el portátil, separado del sandbox de la semana 1.

1. Preparar la carpeta del proyecto y subcarpetas

```
cd ~/proyecto-erp
mkdir -p semana4_odoo_prod_local/{odoo-config,odoo-addons,odoo-data,postgres-data}
```

2. Crear fichero docker-compose.yml dentro de la carpeta `semana4odoo_prod_local`

```
nano docker-compose.yml
```

```

services:
  db:
    image: postgres:15
    container_name: odoo-prod-db
    restart: always
    environment:
      - POSTGRES_DB=postgres
      - POSTGRES_USER=odoo_prod
      - POSTGRES_PASSWORD=odoo_prod_pass
    volumes:
      - ./postgres-data:/var/lib/postgresql/data

  odoo:
    image: odoo:16
    container_name: odoo-prod-app
    restart: always
    depends_on:
      - db
    environment:
      - DB_HOST=db
      - DB_PORT=5432
      - DB_USER=odoo_prod
      - DB_PASSWORD=odoo_prod_pass
    volumes:
      - ./odoo-addons:/mnt/extra-addons
      - ./odoo-config:/etc/odoo
      - ./odoo-data:/var/lib/odoo
    ports:
      - "8069:8069"

```

Nota: levanta dos contenedores: odoo-prod-db (PostgreSQL 15) y odoo-prod-app (Odoo Community 16), conectados entre sí y con volúmenes persistentes para sus datos.

3. Crear fichero odoo.conf

En la carpeta /odoo-conf

```
nano odoo.conf
```

```
[options]
admin_passwd = Clave$123
db_host = db
db_port = 5432
db_user = odoo_prod
db_password = odoo_prod_pass
data_dir = /var/lib/odoo
proxy_mode = True
```

Nota: adminpasswd es la contraseña maestra que pide Odoo cuando creas una base de datos nueva. Los parámetros db son los datos para que Odoo se conecte al contenedor de PostgreSQL, y data_dir es la carpeta donde Odoo guarda sus archivos internos.*

4. Iniciar el despliegue

```
> docker compose up -d          # levanta Odoo y PostgreSQL en segundo plano
> docker ps                     # comprobar que los contenedores están corriendo
> docker compose down          # si se quiere parar el entorno
```

En el asistente de Odoo:

- **Master Password:** Clave\$123
- **Database Name:** vrcardioprodlocal
- **Email:** damaris.anton@educa.madrid.org
- **Password:** Clave\$1

Nota: no se han cargado datos ficticios, ya que en la próxima fase se trabajará con datos reales proporcionados por la empresa.

5. Problemas encontrados

Durante el despliegue del entorno de producción local de Odoo aparecieron varios problemas relacionados con permisos en las carpetas de datos y con el acceso desde el host:

- **Internal Server Error al entrar en Odoo:**

Al acceder a <http://localhost:8069> Odoo mostraba un “Internal Server Error”. Revisando los logs del contenedor aparecía un mensaje del tipo:

```
AssertionError: /var/lib/odoo/sessions: directory is not writable
```

Esto significaba que la carpeta donde Odoo guarda las sesiones (/var/lib/odoo/sessions, que corresponde al volumen odoo-data) no tenía permisos de escritura para el usuario interno de Odoo (UID 101). Para

solucionarlo detuve el contenedor y cambié el propietario de la carpeta:

```
cd ~/proyecto-erp/semana4_odoo_prod_local
docker stop odoo-prod-app
sudo chown -R 101:101 odoo-data
docker start odoo-prod-app
```

Después de este cambio Odoo pudo crear las sesiones correctamente y el error 500 desapareció.

- **Aviso [error opening dir] en odoo-data y postgres-data al usar tree:**

Al listar la estructura de la carpeta del proyecto con tree aparecían líneas como:

```
|— odoo-data  [error opening dir]
   |— postgres-data  [error opening dir]
```

Esto no afectaba al funcionamiento de Odoo ni de PostgreSQL, pero indicaba que mi usuario ubuntu no tenía permisos para ver el contenido de esas carpetas porque pertenecían a otro UID (por ejemplo, el 101 que usa Odoo dentro del contenedor).

```
cd ~/proyecto-erp/semana4_odoo_prod_local
sudo chown -R ubuntu:ubuntu odoo-data postgres-data
```

Configuración robusta de PostgreSQL

Usuario y accesos

En el docker-compose.yml he definido un usuario específico para Odoo:

- **Usuario:** odoo_prod
- **Contraseña:** odooprodpass

Esto se pasa al contenedor de PostgreSQL y también lo uso en odoo.conf y en las variables de entorno de Odoo, para que todo cuadre.

Aislamiento

El puerto 5432 de PostgreSQL no está publicado hacia fuera, solo se usa dentro de la red de Docker entre los contenedores. Así evito que alguien se conecte directamente a la base de datos desde fuera; la única forma de entrar es a través de Odoo.

Persistencia de datos

```
> docker compose down  
> docker compose up -d
```

Los datos de la base se guardan en la carpeta postgres-data. He comprobado que si hago la base *vrcardioprodl* sigue existiendo y puedo entrar con normalidad, así que la parte de persistencia funciona. En el VPS usaré esta misma configuración y allí ya se podrán ajustar parámetros de rendimiento según la RAM que tenga el servidor.

Configuración de HTTPS y restricción de accesos

Aunque esta semana solo he montado el entorno en mi portátil, he dejado pensado cómo voy a proteger el ERP cuando lo pase al VPS, usando Nginx, Let's Encrypt y un firewall.

Nginx como proxy inverso

En el VPS se añadirá Nginx delante de Odoo con la siguiente lógica:

- **Escuchar en los puertos 80 y 443.**
- **Redirigir todo el tráfico HTTP (80) a HTTPS (443).**
- **Reenviar las peticiones HTTPS al contenedor de Odoo (puerto 8069 interno).**

Ejemplo de bloque de configuración planteado:

```

server {
    listen 80;
    server_name erp.vrcardio.example.com;

    location / {
        return 301 https://$host$request_uri;
    }
}

server {
    listen 443 ssl;
    server_name erp.vrcardio.example.com;

    ssl_certificate      /etc/letsencrypt/live/erp.vrcardio.example.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/erp.vrcardio.example.com/privkey.pem;

    location / {
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
        proxy_pass http://odoo:8069;
    }
}

```

Nota: con esto, el usuario siempre entra por HTTPS y Odoo ve las cabeceras correctas de proxy.

Certificados HTTPS con Let's Encrypt

En el VPS se utilizarán certificados gratuitos de Let's Encrypt:

- **Apuntar el dominio del ERP al VPS desde el DNS.**
- **Instalar certbot y ejecutarlo contra Nginx para sacar los certificados SSL/TLS.**
- **Dejar configurada la renovación automática para que los certificados se renueven solos y no caduquen.**

Firewall y restricción de puertos

También voy a limitar qué puertos están abiertos en el VPS, por ejemplo con ufw:

- **Permitir solo:**
 - 22/tcp → SSH para administrar el servidor.
 - 80/tcp → para que Let's Encrypt pueda validar el dominio.
 - 443/tcp → acceso HTTPS de los usuarios al ERP.

- **Bloquear el acceso directo a:**
 - 8069/tcp → puerto interno de Odoo.
 - 5432/tcp → puerto de PostgreSQL.

De esta forma, la aplicación y la base de datos quedan siempre detrás de Nginx y de la red interna de Docker, y desde fuera solo se ve el servicio HTTPS del ERP.

Conclusiones

Durante la cuarta semana se ha conseguido:

1. Montar un entorno de producción local para Odoo Community 16 y PostgreSQL con Docker, separado del sandbox de pruebas.
 2. Dejar definidos el `docker-compose.yml` y el `odoo.conf`, con usuario propio de base de datos y volúmenes para que los datos sean persistentes.
 3. Crear la base de datos `vrcardio_prod_local`, que se mantiene entre reinicios y está preparada para cargar datos reales en las siguientes semanas.
 4. Próximo paso: definir roles y permisos (Administrador, Becario, Ventas, Dirección) y dar de alta el personal en Odoo, iniciando la fase de gestión de usuarios y accesos.
-

Semana 5 – Plan de despliegue de Odoo en AWS y acceso por URL

Introducción

Objetivo de la semana: Dejar preparado el plan de despliegue en la nube de Odoo Community 16 para Spika Tech usando AWS, de forma que el ERP quede accesible mediante una URL con dominio propio (con Nginx como proxy inverso). El despliegue real se hará el lunes, cuando tenga acceso a la cuenta institucional de AWS; esta semana he definido la arquitectura y los pasos exactos (comandos y configuraciones) que seguiré, partiendo de la infraestructura local que monté en la semana 4 con Docker y PostgreSQL.

Diseño de la infraestructura en AWS

Plataforma y recursos

Para el despliegue he elegido Amazon Web Services (AWS) con una instancia EC2 pequeña (1 GB de RAM y 20 GB de disco SSD), suficiente para un entorno piloto de Odoo. AWS permite instalar Ubuntu Server, Docker, docker-compose y Nginx, y ofrece free tier para una instancia pequeña durante el primer año, lo que reduce el coste inicial para la empresa.

Configuración prevista de la instancia EC2:

- **Tipo** → t3.micro / t3a.micro (1 GiB RAM).
- **Disco** → 20 GB SSD.
- **Puertos abiertos** → (Security Group):
 - 22 (SSH)
 - 80 (HTTP)
 - 443 (HTTPS)

Arquitectura objetivo

La arquitectura que quiero tener en AWS es:

- Instancia EC2 con Ubuntu Server 22.04.
- Contenedor Docker odoo-prod-db con PostgreSQL 15.
- Contenedor Docker odoo-prod-app con Odoo 16 Community.
- Carpetas en el host para separar datos y configuración:
 - ~/proyecto-erp/odooaws/odoo-config
 - ~/proyecto-erp/odooaws/odoo-addons
 - ~/proyecto-erp/odooaws/odoo-data
 - ~/proyecto-erp/odooaws/postgres-data
- Nginx instalado en la instancia como proxy inverso, escuchando en 80/443 y reenviando peticiones al Odoo que escucha en 8069.
- Un dominio/subdominio tipo erp.spikatech.com apuntando a la IP de la instancia para el acceso por URL.

Preparación de la instancia EC2 y Docker

Conexión por SSH

Una vez creada la instancia EC2 con Ubuntu, la conexión se hará con la clave .pem:

```
chmod 600 erp-spikatech-key.pem  
ssh -i erp-spikatech-key.pem ubuntu@IP_PUBLICA
```

Instalación de Docker y docker-compose

Dentro de la instancia:

```
sudo apt update  
sudo apt install -y docker.io docker-compose  
sudo usermod -aG docker ubuntu
```

Después saldré y volveré a entrar para que se apliquen los permisos:

```
exit  
ssh -i erp-spikatech-key.pem ubuntu@IP_PUBLICA  
docker ps
```

Despliegue de Odoo en AWS

1. Creación de la estructura de carpetas

Voy a replicar la estructura de producción que usé en la semana 4, pero en AWS:

```
mkdir -p ~/proyecto-erp/odooaws/odoo-config  
mkdir -p ~/proyecto-erp/odooaws/odoo-addons  
mkdir -p ~/proyecto-erp/odooaws/odoo-data  
mkdir -p ~/proyecto-erp/odooaws/postgres-data  
cd ~/proyecto-erp/odooaws
```

Nota: con esto separo configuración y datos para que sean persistentes aunque borre los contenedores.

2. Crear fichero docker-compose.yml dentro de la carpeta odooaws

```
nano docker-compose.yml
```

```

services:
  db:
    image: postgres:15
    container_name: odoo-prod-db
    restart: always
    environment:
      - POSTGRES_DB=postgres
      - POSTGRES_USER=odooprod
      - POSTGRES_PASSWORD=odooprodpass
    volumes:
      - ./postgres-data:/var/lib/postgresql/data

  odoo:
    image: odoo:16
    container_name: odoo-prod-app
    restart: always
    depends_on:
      - db
    environment:
      - DB_HOST=db
      - DB_PORT=5432
      - DB_USER=odooprod
      - DB_PASSWORD=odooprodpass
    volumes:
      - ./odoo-addons:/mnt/extra-addons
      - ./odoo-config:/etc/odoo
      - ./odoo-data:/var/lib/odoo
    ports:
      - "8069:8069"

```

Nota: levanta dos contenedores Docker (PostgreSQL y Odoo) y monta las carpetas del host como volúmenes, igual que en mi entorno de producción local.

3. Crear fichero odoo.conf

En la carpeta /odoo-config

```
nano odoo.conf
```

```
[options]
admin_passwd = Clave123
db_host = db
db_port = 5432
db_user = odoo prod
db_password = odoo prod pass
data_dir = /var/lib/odoo
proxy_mode = True
```

admin_passwd: contraseña maestra para crear bases de datos. db_: datos de conexión al contenedor de PostgreSQL. data_dir: ruta interna de datos de Odoo. proxy_mode = True: deja Odoo listo para trabajar detrás de Nginx.*

4. Levantar los contenedores

En la carpeta del proyecto:

```
cd ~/proyecto-erp/odooaws
docker compose up -d
docker ps
```

Nota: Con esto Odoo y PostgreSQL quedarán levantados en segundo plano en la instancia EC2.

5. Creación de la base de datos

Mientras todavía no tenga dominio, podré entrar (temporalmente) por IP:

- **http://IP_PUBLICA:8069**

En el asistente de Odoo, crearé la base de datos de producción, por ejemplo:

- **Master password:** Clave\$123.
- **Nombre de la base de datos:** vrcardioproductaws.
- **Usuario administrador:** email de admin y contraseña.

Publicación por URL con Nginx y dominio

Instalación y configuración básica de Nginx

Para que el ERP sea accesible por una URL, usaré Nginx como proxy inverso delante de Odoo:

- **Instalación:**

```
sudo apt install -y nginx
```

- **Eliminar la configuración por defecto:**

```
sudo rm /etc/nginx/sites-enabled/default
sudo rm /etc/nginx/sites-available/default
```

- **Crear el archivo `/etc/nginx/sites-available/odoo.conf`:**

```
sudo nano /etc/nginx/sites-available/odoo.conf

```text
upstream odoo {
 server 127.0.0.1:8069;
}

server {
 listen 80;
 server_name erp.spikatech.com;

 proxy_read_timeout 720s;
 proxy_connect_timeout 720s;
 proxy_send_timeout 720s;

 proxy_set_header Host $host;
 proxy_set_header X-Real-IP $remote_addr;
 proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
 proxy_set_header X-Forwarded-Proto $scheme;

 location / {
 proxy_redirect off;
 proxy_pass http://odoo;
 }

 location ~* /web/static/ {
 proxy_cache_valid 200 90m;
 proxy_buffering on;
 expires 864000;
 proxy_pass http://odoo;
 }
}
```

- **Habilitar el sitio y recargar Nginx:**

```
sudo ln -s /etc/nginx/sites-available/odoo.conf /etc/nginx/sites-enabled/odoo.conf
sudo nginx -t
sudo systemctl reload nginx
```

Con esto, cuando el dominio `erp.spikatech.com` apunte a la IP de la instancia, las peticiones a `http://erp.spikatech.com` irán al Odoo del puerto 8069 sin que el usuario lo vea.

---

## HTTPS con Let's Encrypt (planificado)

Una vez que el DNS esté configurado, el siguiente paso será activar HTTPS con Let's Encrypt:

### Instalar certbot

```
sudo apt install -y certbot python3-certbot-nginx
```

### Solicitar el certificado para el dominio:

```
sudo certbot --nginx -d erp.spikatech.com
```

El asistente ajustará la configuración de Nginx para usar HTTPS y, si se selecciona esa opción, redirigir todo el tráfico HTTP a HTTPS. El resultado final será que el ERP esté disponible en:

- **`https://erp.spikatech.com`**

con la conexión cifrada y sin necesidad de exponer el puerto 8069 directamente a Internet.

---

## Conclusiones

Durante la quinta semana se ha conseguido:

1. Dejar preparado el diseño de la infraestructura en AWS (tipo de instancia, puertos, disco, sistema operativo).
2. Crear la estructura de carpetas, el `docker-compose.yml` y el `odoo.conf` que se usarán para levantar Odoo y PostgreSQL en la instancia, siguiendo el modelo que ya funcionó en el entorno local de producción de la semana 4.
3. Definir la configuración prevista de Nginx como proxy inverso y el uso de Let's Encrypt para que el ERP quede publicado mediante una URL propia (`erp.spikatech.com`) y con HTTPS.
4. Próximo paso: aplicar estos comandos y configuraciones en la cuenta de AWS de la empresa para dejar el ERP funcionando en la nube y accesible por URL.

---

# Semana 6 – Despliegue real de Odoo en AWS, HTTPS con Let's Encrypt y diseño de gestión de licencias y proyectos

---

## Introducción

---

**Objetivo de la semana:** Poner en marcha de forma real Odoo Community 16 en la instancia de AWS proporcionada por la empresa, publicándolo en Internet con HTTPS válido y dejando configurada la base para gestionar las licencias de software de VRCardio y los proyectos de implantación y desarrollo dentro de Odoo.

---

## Acceso a la instancia de AWS

---

### Conexión por SSH con la clave PEM

Una vez que la empresa me facilitó el fichero `erp_keys.pem` y el DNS público de la instancia (`ec2-56-228-82-108.eu-north-1.compute.amazonaws.com`), lo primero que hice fue ajustar permisos a la clave y conectarme por SSH desde mi equipo:

```
chmod 400 erp_keys.pem
ssh -i erp_keys.pem ec2-user@ec2-56-228-82-108.eu-north-1.compute.amazonaws.com
```

Tras aceptar la huella del host la primera vez, me conecté como `ec2-user` en un sistema Amazon Linux 2023, desde donde ya he realizado toda la instalación de Docker y el despliegue de Odoo.

## Instalación de Docker y Docker Compose

---

Dentro de la instancia, actualicé paquetes e instalé Docker:

```
sudo dnf update -y
sudo dnf install -y docker
sudo systemctl enable docker
sudo systemctl start docker
sudo usermod -aG docker ec2-user
```

Después salí y volví a entrar por SSH para que se aplicaran los permisos del grupo `docker` y comprobé que Docker estaba operativo con `docker ps`.



Como en Amazon Linux 2023 el paquete docker-compose-plugin no está disponible, instalé Docker Compose descargando el binario y registrándolo como plugin de Docker:

```
sudo mkdir -p /usr/local/lib/docker/cli-plugins/
sudo curl -SL "https://github.com/docker/compose/releases/latest/download/docker-comp
ose-linux-$(uname -m)" -o /usr/local/lib/docker/cli-plugins/docker-compose
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
docker compose version
```

## Despliegue de Odoo y PostgreSQL con Docker

### 1. Creación de la estructura de carpetas

Para mantener ordenados los datos y la configuración, he replicado en la instancia la estructura que ya usé en local:

```
mkdir -p ~/proyecto-erp/odooaws/odoo-config
mkdir -p ~/proyecto-erp/odooaws/odoo-addons
mkdir -p ~/proyecto-erp/odooaws/odoo-data
mkdir -p ~/proyecto-erp/odooaws/postgres-data
cd ~/proyecto-erp/odooaws
```

*Nota: con esto separo la configuración (odoo-config), posibles addons extra (odoo-addons) y los datos persistentes tanto de Odoo (odoo-data) como de PostgreSQL (postgres-data).*

### 2. Crear fichero docker-compose.yml dentro de la carpeta odooaws

```
nano docker-compose.yml
```

```

services:
 db:
 image: postgres:15
 container_name: odoo-prod-db
 restart: always
 environment:
 - POSTGRES_DB=postgres
 - POSTGRES_USER=odooprod
 - POSTGRES_PASSWORD=odooprodpass
 volumes:
 - ./postgres-data:/var/lib/postgresql/data

 odoo:
 image: odoo:16
 container_name: odoo-prod-app
 restart: always
 depends_on:
 - db
 environment:
 - DB_HOST=db
 - DB_PORT=5432
 - DB_USER=odooprod
 - DB_PASSWORD=odooprodpass
 volumes:
 - ./odoo-addons:/mnt/extra-addons
 - ./odoo-config:/etc/odoo
 - ./odoo-data:/var/lib/odoo
 ports:
 - "8069:8069"

```

*Nota: este fichero levanta dos contenedores: uno con PostgreSQL 15 y otro con Odoo 16 Community, utilizando las carpetas del host como volúmenes para que los datos no se pierdan al recrear contenedores.*

### 3. Crear fichero odoo.conf

En la carpeta /odoo-config

```
nano odoo.conf
```

```
[options]
 admin_passwd = Qen01,6xc(7Xl8!L
 db_host = db
 db_port = 5432
 db_user = odoo prod
 db_password = odoo prod pass
 data_dir = /var/lib/odoo
 proxy_mode = True
```

*Nota: adminpasswd será la contraseña maestra para la creación de bases de datos, y los parámetros db\* apuntan al contenedor de PostgreSQL definido en docker-compose.yml. El datadir coincide con el volumen montado en odoo-data y proxymode = True deja preparado Odoo para trabajar en el futuro detrás de un proxy inverso.*

## 4. Levantar los contenedores

En la carpeta del proyecto:

```
cd ~/proyecto-erp/odooaws
docker compose up -d
docker ps
```

## 5. Resolución del error 500

Al probar desde la propia instancia inicialmente obtenía un error 500. Revisando los logs del contenedor de Odoo:

```
curl http://localhost:8069
docker logs --tail=100 odoo-prod-app
```

encontré un **Permission denied: '/var/lib/odoo/sessions'**, lo que indicaba que el usuario interno de Odoo no tenía permisos para escribir en el volumen odoo-data. Para solucionarlo, probé a cambiar el propietario del directorio y, abrí permisos temporalmente para que Odoo pudiera crear la carpeta de sesiones:

```
docker compose stop odoo
sudo chmod -R 777 odoo-data
docker compose up -d odoo
```

Después de este ajuste, el acceso a **http://localhost:8069/web/database** devolvió el HTML de la pantalla de selección/creación de base de datos.

## 6. Creación de la base de datos

En el asistente de Odoo:

- **Master Password:** Qen01,6xc(7Xl8!L
  - **Database Name:** vrcardio
  - **Email:** admin.its@spikatech.com
  - **Password:** c19U{o8PT|Z()~3R
- 

## Publicación del ERP con Nginx y subdominio erp.spikatech.com

---

### Instalación y configuración básica de Nginx

#### 1. Instalación de Nginx para dejarlo activo

```
sudo dnf install -y nginx
sudo systemctl enable nginx
sudo systemctl start nginx
```

#### 2. Crear fichero odoo.conf

En /etc/nginx/conf.d/odoo.conf he creado una configuración para que Nginx escuche en 80/443, redirija HTTP a HTTPS y actúe como proxy inverso hacia Odoo en 127.0.0.1:8069.

```
sudo nano /etc/nginx/conf.d/odoo.conf
```

```

server {
 listen 80;
 server_name erp.spikatech.com;

 return 301 https://$host$request_uri;
}

server {
 listen 443 ssl;
 server_name erp.spikatech.com;

 ssl_certificate /etc/letsencrypt/live/erp.spikatech.com/fullchain.pem;
 ssl_certificate_key /etc/letsencrypt/live/erp.spikatech.com/privkey.pem;

 location / {
 proxy_set_header Host $host;
 proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
 proxy_set_header X-Forwarded-Proto https;
 proxy_pass http://127.0.0.1:8069;
 }
}

```

### 3. Comprobar la configuración y recargar Nginx:

```

sudo nginx -t
sudo systemctl reload nginx

```

### Creación del subdominio y pruebas de puertos

La empresa ha creado el subdominio erp.spikatech.com en su dominio, con un registro DNS tipo A apuntando a la IP pública 56.228.82.108 de la instancia. Desde mi equipo lo he comprobado con:

```
nslookup erp.spikatech.com
```

*Nota: se obtiene Address: 56.228.82.108.*

Para verificar qué puertos se ven desde Internet he usado nmap:

```
nmap -p 22,80,443,8069,5432 erp.spikatech.com
```

El resultado muestra:

PORT	STATE	SERVICE
22/tcp	open	ssh
80/tcp	open	http
443/tcp	open	https
8069/tcp	filtered	unknown
5432/tcp	filtered	postgresql

*Nota: esto confirma que solo están expuestos 22, 80 y 443, mientras que los puertos internos de Odoo (8069) y PostgreSQL (5432) permanecen filtrados y no se puede acceder directamente a ellos desde Internet.*

Desde el navegador ya es posible acceder a Odoo con la URL: **https://erp.spikatech.com**

## Configuración de HTTPS con Let's Encrypt (Certbot)

Para cumplir el requisito de **Configuración de HTTPS Let's Encrypt y restricción de accesos** de la semana 4 he sustituido el certificado autofirmado por un certificado gratuito de Let's Encrypt usando Certbot.

### Instalación de Certbot y emisión del certificado

En la instancia he instalado Certbot y el plugin de Nginx, y luego he ejecutado Certbot indicando el subdominio:

```
sudo dnf install -y certbot python3-certbot-nginx
sudo certbot --nginx -d erp.spikatech.com
```

Durante el asistente he introducido el correo **admin.its@spikatech.com**, aceptado los términos de servicio y seleccionado la opción de redirigir todo el tráfico HTTP a HTTPS.

Certbot ha modificado automáticamente la configuración de Nginx para usar los certificados de Let's Encrypt ubicados en `/etc/letsencrypt/live/erp.spikatech.com/` y ha configurado la renovación automática.

Finalmente, he probado la renovación en modo simulación:

```
sudo certbot renew --dry-run
```

Desde el navegador, al acceder de nuevo a **https://erp.spikatech.com**, el candado aparece como seguro y el certificado se identifica como emitido por Let's Encrypt para `erp.spikatech.com`, sin avisos de

seguridad.

---

## Gestión de licencias de software (Inventario / Suscripciones)

---

Además de la parte de sistemas, esta semana he dejado preparada la configuración funcional para gestionar las licencias de software de VRCardio usando el módulo de Inventario (y opcionalmente Suscripciones) de Odoo.

### Licencias como productos de servicio

La idea es modelar cada tipo de licencia como un producto de tipo servicio vendible, asociado a una categoría específica de “Licencias VR”, como por ejemplo el usado en la semana 1:

Ruta: Inventario → Productos → Productos → Nuevo

- **Tipo de producto:** Servicio
- **Categoría:** Licencias VR (creada para agrupar las licencias)
- **Impuestos:** IVA correspondiente
- **Campos adicionales:** descripción técnica de la licencia, duración, etc

### Control de fechas de caducidad y asignación a clientes

El objetivo es poder controlar la fecha de caducidad de cada licencia y a qué cliente/hospital está asignada.

A nivel de diseño:

- Cada licencia contratada se reflejará en Odoo como una línea de pedido de venta y posteriormente como una factura asociada al producto de licencia correspondiente.
- La fecha de caducidad se controlará mediante campos de fecha en la ficha de la licencia o mediante el módulo de Suscripciones (si se activa), que permite gestionar renovaciones automáticas y periodos de validez.
- Mediante la relación entre pedido/factura y cliente, se podrá ver qué licencias tiene cada hospital y cuándo expiran para planificar renovaciones.

En esta semana he dejado documentado este modelo de datos y el flujo de trabajo, a la espera de importar las licencias reales mediante CSV cuando reciba las plantillas de la empresa.

---

## Gestión de proyectos y seguimiento de tareas (Kanban, horas)

---

### Configuración del módulo de Proyectos

El flujo que he diseñado es:

- Crear un proyecto por cada implantación o cliente.
- Definir columnas Kanban que representen los estados de las tareas (Pendiente, En curso, En revisión, Completada).
- Dentro de cada proyecto, crear tareas para las actividades principales: instalación, configuración, formación, soporte, desarrollo de nuevas funcionalidades, etc.

Esto permite que el equipo tenga una visión clara del estado de los trabajos en curso y que dirección vea la carga de trabajo por cliente.

## Imputación de horas en tareas

Odoo permite registrar el tiempo dedicado por cada usuario a las tareas del proyecto. A nivel de diseño, la idea es:

- Cada técnico inicia y registra el tiempo en las tareas que tenga asignadas, indicando fecha, duración y descripción breve de lo realizado.
- Desde el proyecto se puede ver el total de horas por tarea, por empleado y por proyecto, lo que ayuda a controlar el esfuerzo invertido en cada implantación o desarrollo vinculado a las licencias vendidas.

---

## Conclusiones

Durante la sexta semana se ha conseguido:

1. Dejar desplegado en la instancia de AWS un entorno de producción con Odoo y PostgreSQL en Docker, correctamente configurado y protegido detrás de Nginx para que no se expongan directamente los puertos internos.
2. Configurar el acceso al ERP mediante el subdominio `erp.spikatech.com`, usando Nginx como proxy inverso y activando HTTPS con certificados válidos de Let's Encrypt, de forma que ahora el acceso es seguro y sin avisos de certificado en el navegador.
3. Definir cómo se gestionarán las licencias de software y los proyectos en Odoo, utilizando el módulo de Inventario/Suscripciones para las licencias y el módulo de Proyectos con vista Kanban e imputación de horas por tarea.
4. Próximo paso: trabajar la parte de CRM y ventas sobre este Odoo de producción, configurando el pipeline de oportunidades y la generación de presupuestos y facturas con los datos de VRCardio.



# Semana 7 - Configuración del CRM y personalización de presupuestos/facturas con logo y datos fiscales

---

## Introducción

---

**Objetivo de la semana:** Adaptar el módulo de CRM/Ventas de Odoo al proceso comercial de Spika Tech, definiendo un pipeline con las etapas Lead → Contacto → Demo → Cierre, y personalizar las plantillas PDF de presupuestos y facturas para que incluyan el logotipo corporativo y los datos fiscales y legales de la empresa, según el documento del proyecto.

---

## Configuración del pipeline de CRM (Lead → Contacto → Demo → Cierre)

---

### Acceso al CRM y revisión del flujo por defecto

Partiendo del Odoo de producción desplegado en AWS, he accedido al módulo CRM desde la barra superior de aplicaciones. La vista principal muestra el tablero de oportunidades en formato Kanban, con las columnas estándar “Nuevo”, “Calificado”, “Propuesta” y “Ganado”, además del botón “NUEVO” para crear oportunidades. Este flujo es genérico y no refleja todavía las fases concretas que necesita Spika Tech para gestionar su ciclo de ventas.

### Redefinición de etapas del pipeline

Para adaptar el sistema al proceso comercial definido en el proyecto, he entrado en CRM → Ventas → Mi flujo y he editado las etapas existentes. A partir de las columnas por defecto (“Nuevo”, “Calificado”, “Propuesta”, “Ganado”), he renombrado y reordenado las etapas hasta dejar las siguientes fases:

- **Lead:** registro de oportunidades iniciales, por ejemplo hospitales o clínicas que muestran interés por VRCardio o solicitan información.
- **Contacto:** oportunidades en las que ya se ha realizado un primer contacto comercial (llamada, correo, reunión) y se ha validado que hay interés real.
- **Demo:** fase en la que se ha propuesto o realizado una demostración de la solución, ya sea presencial o remota.
- **Cierre:** etapa final del embudo, donde se negocian condiciones, precios y fechas de implantación con el objetivo de cerrar el acuerdo.

Tras guardar los cambios, el tablero de CRM pasa a mostrar las columnas Lead, Contacto, Demo y Cierre en lugar de los nombres genéricos iniciales. Esto deja preparado el entorno para trabajar con datos reales cuando la empresa los facilite.

---

## Configuración de la compañía: logo y datos fiscales

---

### Edición de la ficha de compañía en Odoo

En paralelo al CRM, he trabajado en la configuración de la compañía dentro de Odoo, ya que estos datos son la base de los documentos de ventas (presupuestos y facturas). Desde el módulo Ajustes he accedido a Gestionar compañías y he editado el registro principal “My Company” para que refleje los datos reales de Spika Tech.

En esta ficha he dejado configurado:

- **Nombre de la empresa:** SPIKA TECH, S.L
- **Dirección:** c/ Alcalá de Guadaira, nº 6, 6ºA Derecha, 28018 MADRID, España
- **NIF:** B87386900
- **Moneda:** EUR
- **Correo electrónico:** admin.its@spikatech.com
- **Sitio web:** <http://spikatech.com>

Además, he cargado el logotipo corporativo de Spika Tech en la parte superior de la ficha de compañía. Gracias a esto, el logo ya aparece tanto en la pantalla de login de Odoo como en la interfaz una vez dentro del ERP, y se reutiliza también en los presupuestos y facturas en PDF.

---

## Personalización del diseño de presupuestos y facturas (PDF)

---

### Configuración del diseño de documentos en Odoo

Una vez que la compañía ya tenía el logo y los datos fiscales bien puestos, he usado el asistente de Diseño de documento de Odoo para ajustar cómo salen los PDFs de presupuestos y facturas. Para ello he ido a Ajustes → Configurar diseño de documento y he:

- Elegido un tema (por ejemplo Light)
- Revisado que use el logotipo de Spika Tech
- Comprobado en **Detalles de la compañía** que aparecen los mismos datos que en la ficha de empresa:
  - Razón social: SPIKA TECH, S.L.
  - Dirección completa en Madrid
  - País: España

Con esto me aseguro de que en la cabecera del PDF salga el emisor con nombre, NIF y domicilio.

## Inserción del texto legal en el pie de página

Para que el PDF se parezca al modelo del proyecto, he rellenado el Pie de página del asistente con:

- **Primera línea:** correo y web de la empresa  
contact@spikatech.com - www.spikatech.com
- **Segunda línea:** el texto legal completo del documento del proyecto, con domicilio social, NIF y datos del Registro Mercantil de Madrid.

Así, todos los presupuestos y facturas que genera Odoo llevan abajo la misma coetilla legal que el PDF de referencia.

## Resultado final

Después de estos cambios, los documentos quedan así:

- Logotipo de Spika Tech en la parte superior.
- Razón social, NIF y dirección visibles en la cabecera.
- Texto legal completo en el pie de página.

No es exactamente igual que el PDF original (el asistente de Odoo tiene sus límites), pero el diseño es muy parecido y, sobre todo, cumple con lo que pide el proyecto usando solo configuración, sin tocar código.

---

## Conclusiones

Durante la séptima semana se ha conseguido:

1. Adaptar el módulo de CRM de Odoo al flujo comercial de VRCardio/Spika Tech, sustituyendo las etapas genéricas por el pipeline Lead → Contacto → Demo → Cierre.
  2. Configurar la ficha de la compañía con los datos fiscales reales de Spika Tech y el logotipo corporativo, que ya aparece en el login y dentro del ERP.
  3. Personalizar las plantillas PDF de presupuestos y facturas para que incluyan el logo, los datos fiscales y el texto legal completo que la empresa utiliza en sus documentos.
  4. Próximo paso: investigar la API de Odoo y crear un pequeño script en Python que se conecte al ERP desde fuera y lea algún dato sencillo (por ejemplo, un cliente u oportunidad) como base para futuras integraciones.
-

# Semana 8 – Pruebas de integración con Sistemas (API)

---

## Introducción

---

**Objetivo de la semana:** Investigar la API de Odoo (XML-RPC) y desarrollar un primer script en Python que se conecte desde fuera al ERP de producción desplegado en AWS, se autentique contra la base de datos vrcardio y lea algunos datos reales del sistema (contactos). Partiendo del Odoo 16 Community que ya dejé instalado en la instancia de AWS, publicado con Nginx y HTTPS bajo el subdominio `erp.spikatech.com`, en esta semana he dado el primer paso hacia la integración externa del ERP, preparando un usuario técnico específico y construyendo un pequeño “Hola Mundo” de integración que demuestra que se pueden consultar datos desde scripts o aplicaciones externas.

---

## Creación de un usuario técnico para la API

---

Antes de escribir el script, he creado un usuario específico para las integraciones, con el fin de no utilizar la cuenta de administrador general y poder controlar mejor los accesos.

Los pasos que he seguido han sido:

1. Acceder a Odoo en la URL **`https://erp.spikatech.com`**, seleccionando la base de datos vrcardio e iniciando sesión con el usuario administrador.
2. Ir al módulo Ajustes → Usuarios y empresas → Usuarios → Nuevo para crear un usuario llamado “Usuario API”.
  - **Correo electrónico:** `api.user@spikatech.com`
  - **Contraseña:** `5NjF65G!|9TdW;R*`
3. Guardar el usuario y comprobar que puede iniciar sesión en la web, cerrando sesión con el administrador e iniciando con `api.user@spikatech.com` y su contraseña.

Con esto queda preparado un usuario de servicio específico para la API, que en el futuro se podrá restringir o deshabilitar sin afectar al resto de usuarios de la organización.

---

## Investigación básica de la API XML-RPC de Odoo

---

Odoo proporciona una API estándar basada en XML-RPC que permite realizar operaciones sobre los modelos internos del ERP (como `res.partner` para contactos o `crm.lead` para oportunidades) desde aplicaciones externas, siempre que se disponga de la URL, el nombre de la base de datos y las credenciales de un usuario válido.

La API se divide en dos endpoints principales:

- **<https://erp.spikatech.com/xmlrpc/2/common>**: utilizado para autenticarse mediante la función `authenticate(db, username, password, {})`, que devuelve un identificador de usuario (`uid`) si el login es correcto o `False` si falla.
- **<https://erp.spikatech.com/xmlrpc/2/object>**: usado para invocar métodos sobre los modelos mediante la función `executew`, *que permite hacer operaciones como searchread* (buscar y leer registros), `create`, `write`, etc.

Para la práctica de esta semana he decidido trabajar con el modelo `res.partner`, que es el que Odoo utiliza para almacenar contactos, clientes y proveedores, ya que es un modelo sencillo y siempre tiene algunos datos creados por defecto (empresa, administradores, etc.).

---

## Desarrollo del script “Hola Mundo” en Python

---

El siguiente paso ha sido desarrollar un script en Python 3 que se conecte a la instancia de Odoo en AWS, se autentique con el usuario técnico creado y lea algunos contactos mediante la API XML-RPC.

### Ubicación y entorno

He creado el script directamente en la instancia de AWS, dentro de la carpeta del proyecto del ERP donde ya tenía el `docker-compose.yml` y el resto de archivos de despliegue de Odoo:

```
cd ~/proyecto-erp/odooaws
nano odoo_api_test.py
```

La instancia ya cuenta con Python 3 instalado, por lo que no ha sido necesario añadir dependencias externas; para trabajar con XML-RPC he usado el módulo estándar `xmlrpc.client` que viene con Python.

## Código del script

```
import xmlrpc.client

url = "https://erp.spikatech.com"
db = "vrcardio"
username = "api.user@spikatech.com"
password = "5NjF65G!|9TdW;R*"

common = xmlrpc.client.ServerProxy(f"{url}/xmlrpc/2/common")
uid = common.authenticate(db, username, password, {})

print("UID devuelto por Odoo:", uid)
if not uid:
 raise SystemExit("Error: no se ha podido autenticar (revisa usuario/contraseña/ba
se de datos)")

models = xmlrpc.client.ServerProxy(f"{url}/xmlrpc/2/object")

partners = models.execute_kw(
 db,
 uid,
 password,
 "res.partner",
 "search_read",
 [],
 {"fields": ["name", "email"], "limit": 5},
)

print("Contactos leídos desde Odoo:")
for p in partners:
 print("-", p["name"], "-", p.get("email", "sin email"))
```

*Nota: el script realiza tres pasos principales: autenticarse en Odoo, conectarse al endpoint de objetos y ejecutar una consulta `search_read` sobre el modelo `res.partner` para recuperar los primeros cinco contactos, mostrando por pantalla su nombre y correo electrónico.*

## Pruebas de ejecución y resultados obtenidos

Una vez guardado el archivo `odooapitest.py`, he ejecutado el script desde la misma carpeta del proyecto:

```
cd ~/proyecto-erp/odooaws
python3 odoo_api_test.py
```

El script ha mostrado por consola el uid devuelto por Odoo ("UID devuelto por Odoo: 7") seguido de un listado de contactos con su nombre y correo electrónico:

```
Contactos leídos desde Odoo:
- Administrator - admin@example.com
- Administrator - admin.its@spikatech.com
- SPIKA TECH, S.L. - admin.its@spikatech.com
- Usuario API - api.user@spikatech.com
```

Estos resultados demuestran que el script es capaz de conectarse a la instancia de Odoo desplegada en AWS, autenticarse como el usuario técnico y leer datos reales del modelo `res.partner` a través de la API XML-RPC.

---

## Conclusiones

Durante la octava semana se ha conseguido:

1. Crear un usuario técnico de servicio (Usuario API) para acceder al ERP sin usar la cuenta de administrador.
2. Entender la API XML-RPC de Odoo y cómo autenticarse y consultar modelos desde fuera.
3. Desarrollar y ejecutar un script “Hola Mundo” en Python que se conecta a Odoo en AWS y lee contactos reales de `res.partner`.
4. Próximo paso: reutilizar esta integración para trabajar con otros modelos del negocio y empezar a automatizar intercambios de datos con el ERP.

---

## Semana 9 – Importación de Datos

---

### Introducción

**Objetivo de la semana:** preparar y ejecutar la importación de datos importantes en el Odoo de producción (base de datos `vrCARDIO`): clientes, proveedores, productos/licencias y empleados, comprobando que los registros quedan completos para usarlos en ventas, compras y RRHH. Partiendo del Odoo 16 Community desplegado en la instancia de AWS y publicado con Nginx y HTTPS en <https://erp.spikatech.com>, en esta semana he diseñado las plantillas CSV y he desarrollado un script en Python que se conecta a Odoo mediante la API XML-RPC usando un usuario técnico de servicio (Usuario API) para realizar la carga masiva.

# Limpieza – Preparación de los CSV de proveedores y productos (más empleados)

## Datos que se van a importar

Antes de tocar Odoo he definido qué datos tenía que tener preparados:

- **Proveedores:** empresas que suministran hardware, servicios cloud, etc.
- **Productos:** elementos que se van a vender desde el ERP (licencias de software, servicios asociados, etc).
- **Empleados:** personas de VRCardio que van a usar el sistema o aparecer en ventas y proyectos.

En Odoo estos datos se guardan en estos modelos estándar: `res.partner` para clientes y proveedores, `product.template` para productos/licencias y `hr.employee` para empleados, así que he diseñado los CSV siguiendo esos campos.

## CSV de proveedores

Para los proveedores he creado un fichero separado **proveedores\_plantilla.csv** con la misma estructura que el de clientes, pero con datos y etiquetas de proveedor.

```
name,is_company,company_type,street,city,zip,country_id,vat,phone,email,category_id
```

- CIF/NIF de cada proveedor en vat.
- País correcto en country\_id para que Odoo lo encuentre.
- Etiquetas claras en category\_id (“Proveedor;Tecnología”, “Proveedor;Cloud”) para distinguirlos de los clientes.

## CSV de productos

Para los productos he preparado **productos\_plantilla.csv**, usando el diseño de la semana 6: licencias como servicios en la categoría “Licencias VR”, con su precio e impuestos.

```
name,default_code,type,sale_ok,purchase_ok,list_price,taxes_id,categ_id,description_sale
```

- type="service" para indicar que es un servicio.
- saleok=True y purchaseok=False porque se vende pero no se compra.
- default\_code como código interno tipo SKU.
- list\_price como precio de venta sin impuestos.
- taxes\_id con el nombre del impuesto (“IVA 21%”) que ya está creado en Odoo.
- categ\_id="Licencias VR" para agrupar todas las licencias.



- `description_sale` como texto que se verá en presupuestos y facturas.

De momento la empresa todavía no me ha enviado el fichero de productos relleno, así que esta semana solo he dejado preparada la plantilla `licencias_plantilla.csv` a la espera de importarla en cuanto me la pasen.

## CSV de empleados

Para empleados he preparado el fichero **`empleados_plantilla.csv`**, con los campos básicos del modelo de empleados.

```
name,work_email,work_phone,job_title,department_id
```

- `name` con el nombre completo.
- `workemail` y `workphone` como contacto de trabajo.
- `job_title` para el puesto (“Responsable de Ventas”).
- `department_id` con el nombre del departamento (“Ventas”) ya creado en Odoo.

He comprobado que los correos de trabajo no se repiten y que cada empleado tiene un departamento asignado.

---

## Importación – Carga masiva con script y usuario API

---

### Uso de un usuario API

Para hacer la importación no he usado el usuario administrador general, sino un usuario técnico de servicio (“Usuario API”) que ya había creado para integraciones en la semana 8. De esta forma separo las tareas automáticas (scripts) del uso normal del ERP: si en algún momento hay que bloquear los accesos desde scripts, basta con desactivar este usuario sin afectar al resto de usuarios ni al admin.

### Conexión del script a Odoo

He creado el script **`importodoodata.py`** en la instancia de AWS, dentro de la carpeta del proyecto, reutilizando el mismo esquema de conexión XML-RPC que en el “Hola Mundo” de la semana 8.

```

import csv
import xmlrpc.client

url = "https://erp.spikatech.com"
db = "vrcardio"
username = "api.user@spikatech.com"
password = "*****" # contraseña del usuario API

common = xmlrpc.client.ServerProxy(f"{url}/xmlrpc/2/common")
uid = common.authenticate(db, username, password, {})
if not uid:
 raise SystemExit("Error autenticando contra Odoo")

models = xmlrpc.client.ServerProxy(f"{url}/xmlrpc/2/object")

```

*Nota: con esto el script se autentica en la base de datos vrcardio con el usuario API y queda listo para crear registros en los modelos necesarios.*

## Importación de proveedores

Para los proveedores, el script abre **proveedores\_plantilla.csv** y crea contactos en res.partner para cada fila.

```

def import_proveedores():
 with open("proveedores_plantilla.csv", newline="", encoding="utf-8") as f:
 reader = csv.DictReader(f)
 for row in reader:
 vals = {
 "name": row["name"],
 "is_company": row["is_company"] == "True",
 "company_type": row["company_type"],
 "street": row["street"],
 "city": row["city"],
 "zip": row["zip"],
 "vat": row["vat"],
 "phone": row["phone"],
 "email": row["email"],
 }
 models.execute_kw(db, uid, password, "res.partner", "create", [vals])

```

*Nota: tras ejecutar esta función, en el módulo Contactos aparecen los proveedores importados con sus nombres, ciudades y correos.*

## Importación de empleados

Para los empleados, el script usa **empleados\_plantilla.csv** y crea registros en hr.employee.

```
def import_empleados():
 with open("empleados_plantilla.csv", newline="", encoding="utf-8") as f:
 reader = csv.DictReader(f)
 for row in reader:
 vals = {
 "name": row["name"],
 "work_email": row["work_email"],
 "work_phone": row["work_phone"],
 "job_title": row["job_title"],
 }
 models.execute_kw(db, uid, password, "hr.employee", "create", [vals])
```

*Nota: después de lanzar esta parte del script, en el módulo Empleados aparecen las tarjetas de los empleados con su correo, teléfono y puesto, como se ve en la vista Kanban.*

## Ejecución del script y comprobaciones

En la instancia de AWS he ejecutado el script desde la carpeta del proyecto:

```
cd ~/proyecto-erp
python3 import_odoo_data.py
```

El script muestra un mensaje de fin de importación y, a continuación, he verificado en la interfaz web de Odoo que:

- En **Contactos** se ven los proveedores importados con su información básica.
- En **Empleados** se ven los empleados importados con su correo, teléfono y puesto.

## Conclusiones

Durante la semana 9 se ha conseguido:

1. Diseñar y preparar las plantillas CSV de proveedores, productos y empleados, ajustadas a los modelos estándar de Odoo.

2. Completar estas plantillas con datos coherentes (direcciones, países, teléfonos, correos, categorías) realizando una limpieza básica para evitar errores en la importación.
3. Desarrollar un script en Python que se conecta a Odoo mediante XML-RPC utilizando un usuario técnico de servicio (Usuario API) y que importa de forma masiva los proveedores y empleados definidos en los CSV.
4. Próximo paso: configurar el sistema de copias de seguridad automáticas de la base de datos vrcardio y la sincronización de estos backups con el NAS Synology de la empresa.

---

## Semana 10 - Estrategia de Backups y Recuperación

---

### Introducción

---

**Objetivo de la semana:** automatizar la generación diaria de dumps de la base de datos, almacenarlos de forma ordenada con política de retención y dejar preparada la futura integración con el NAS Synology para disponer de copias externas.

---

### Sistema de backup

---

#### Backup manual inicial de la base de datos

Como primer paso he realizado un backup manual de la base de datos del ERP para comprobar el uso de pg\_dump desde el contenedor Docker de PostgreSQL y validar la ruta de almacenamiento de las copias. Para ello he ejecutado:

```
docker exec -t odoo-prod-db pg_dump -U odooProd -d postgres > ~/proyecto-erp/backups/vr
cardio_$(date +%Y%m%d%H%M).sql
ls -lh ~/proyecto-erp/backups
```

Con este comando se genera un fichero **vrcardio\_YYYYMMDDHHMM.sql** dentro del directorio ~/proyecto-erp/backups, que contiene un volcado completo de la base de datos postgres utilizada por el ERP. El listado me ha permitido comprobar que el archivo se crea correctamente y que el tamaño es coherente con la información almacenada en el sistema.

#### Creación del script de backup automatizado

He creado un script de bash encargado de generar las copias de seguridad y de gestionar la retención de los ficheros antiguos. Los pasos seguidos han sido:

```
mkdir -p scripts
nano scripts/backup_vrcardio.sh
chmod +x scripts/backup_vrcardio.sh
scripts/backup_vrcardio.sh
ls -lh ~/proyecto-erp/backups
```

El contenido final del script **backup\_vrcardio.sh** es el siguiente:

```
#!/bin/bash
FECHA=$(date +%Y%m%d%H%M)

mkdir -p /home/ec2-user/proyecto-erp/backups

docker exec -t odoo-prod-db \
 pg_dump -U odoo prod -d vrcardio > /home/ec2-user/proyecto-erp/backups/vrcardio_${FECHA}.sql

find /home/ec2-user/proyecto-erp/backups -type f -name "vrcardio_*.sql" -mtime +7 -delete
```

*Nota: crea el directorio de backups del proyecto si aún no existe. Ejecuta pg\_dump dentro del contenedor odoo-prod-db utilizando el usuario odoo prod y vuelca la base de datos vrcardio a un fichero con marca de tiempo. Elimina automáticamente las copias de seguridad con más de 7 días de antigüedad mediante find ... -mtime +7 -delete, manteniendo así controlado el espacio en disco. Nota: tras hacerlo ejecutable con chmod +x, he lanzado el script a mano y he verificado con ls -lh que se generan nuevos archivos .sql en el directorio de backups con el formato esperado. Nota: no es necesario generar varios backups separados por “servicios” de Odoo, ya que toda la información del ERP (clientes, proyectos, licencias, empleados, etc.) se guarda en la misma base de datos **vrcardio**. Con este **pg\_dump** diario sobre **vrcardio** se obtiene una copia completa de todos los datos del sistema; en caso de desastre, bastaría con restaurar esta base de datos y volver a desplegar los contenedores para recuperar el ERP.*

## Programación del backup diario con cron

Una vez validado el script, he configurado una tarea programada con cron para que el backup se ejecute de forma automática todos los días a una hora fija. En la instancia de AWS se utiliza el servicio crond (paquete cronie) para gestionar las tareas programadas del usuario ec2-user.

Primero he instalado y activado el servicio:

```
sudo dnf install -y cronie
sudo systemctl enable crond
sudo systemctl start crond
sudo systemctl status crond
```

A continuación, he editado el crontab del usuario para añadir la tarea:

```
crontab -e
```

Dejando la siguiente línea:

```
0 3 * * * /home/ec2-user/proyecto-erp/scripts/backup_vrcardio.sh >> /home/ec2-user/back
up.log 2>&1
```

Con esta configuración:

- El script de backup se ejecuta todos los días a las 03:00.
- Se llama al script usando la ruta absoluta dentro del proyecto.
- Toda la salida (tanto estándar como de error) se redirige al fichero `/home/ec2-user/backup.log`, que sirve como registro para revisar si los backups se han realizado correctamente.

Por último, he comprobado que la entrada se ha guardado correctamente con:

```
crontab -l
```

---

## Integración con NAS Synology

El objetivo final es que, además de las copias locales en la instancia de AWS, los backups de la base de datos queden también guardados en el NAS Synology del centro. Para ello he diseñado una integración basada en `rsync` y he empezado a probar la conexión con el usuario `odoo` que me han facilitado.

### Diseño de la integración

La idea es la siguiente:

- Que el NAS Synology permita acceso remoto desde mi instancia de AWS (por SSH/SFTP y `rsync`).
- Desde la EC2, sincronizar el contenido de `/home/ec2-user/proyecto-erp/backups` con una carpeta del NAS (en principio la carpeta compartida `odoo` que me han indicado).
- Añadir el comando de sincronización al final del script `backup_vrcardio.sh` o crear una tarea `cron` adicional para que, después de generar el dump local, se copie automáticamente al NAS.

El comando que tengo preparado para cuando la conexión funcione es:

```
rsync -avz /home/ec2-user/proyecto-erp/backups/ \
odoo@dreamtechnologycl.fr3.quickconnect.to:/odoo/
```

## Pruebas de conexión con el NAS

Con el usuario `odoo` y la URL de QuickConnect que me han dado, he intentado conectarme desde la instancia de AWS de tres formas distintas:

```
ssh odoo@dreamtechnologycl.fr3.quickconnect.to
sftp odoo@dreamtechnologycl.fr3.quickconnect.to
rsync -avz /home/ec2-user/proyecto-erp/backups/ \
odoo@dreamtechnologycl.fr3.quickconnect.to:/odoo/
```

En los tres casos, al introducir la contraseña de `odoo`, el NAS responde siempre con:

```
Permission denied (password).
```

La integración con Synology queda ya diseñada y con los comandos preparados, pero pendiente de solucionar el problema de los permisos necesarios para el usuario `odoo`. Mientras tanto, el sistema de copias de seguridad automáticas en la instancia de AWS funciona correctamente y genera los dumps diarios con una política de retención de 7 días.

## Conclusiones

Durante la semana 10 se ha conseguido:

1. Definir y poner en marcha un sistema de copias de seguridad automatizadas de la base de datos del ERP en la instancia de AWS, con generación diaria de dumps y política de retención de 7 días.
2. Implementar un script de backup ( `backup_vrcardio.sh` ) integrado con `cron`, que ejecuta `pg_dump` dentro del contenedor de PostgreSQL, genera los ficheros con marca de tiempo y registra la ejecución en un fichero de log.
3. Diseñar la integración con el NAS Synology utilizando rsync y realizar pruebas de conexión desde la instancia de AWS, quedando la copia externa pendiente.

## Semana 11 - Manuales de Procedimiento

**Objetivo de la semana:** redactar dos guías rápidas para el staff de VRCardio (“Cómo dar de alta una nueva licencia” y “Cómo imputar horas a un proyecto”) y publicarlas dentro del Docusaurus del Proyecto 1, usando el repositorio existente.

---

## Preparación del entorno de documentación

---

Para poder editar la documentación he preparado primero el entorno de Docusaurus del Proyecto 1 en la instancia de AWS:

- He clonado el repositorio de documentación proporcionado por mi compañera con `git clone`, obteniendo la carpeta del proyecto con todo el sitio de Docusaurus.
  - Dentro de esa carpeta he ejecutado `npm install` para que npm lea el `package.json` e instale todas las dependencias necesarias (React, Docusaurus, etc.), dejándolo listo para desarrollo.
  - A continuación he arrancado el servidor de desarrollo con `npm run start` y he comprobado en `http://localhost:3000/docs` que se cargaban correctamente las secciones ya existentes (ISO 13485 e ISO 27001) antes de añadir las nuevas guías.
- 

## Contenido de las guías rápidas para el staff de VRCardio

---

He creado una nueva carpeta `docs/vrcardio` y dentro de ella dos ficheros Markdown que actúan como “manuales de procedimiento” breves:

1. Guía “Cómo dar de alta una nueva licencia” ( `docs/vrcardio/licencias_alta.md` )
    - Explica a personal de administración/soporte cómo crear una licencia VRCardio en Odoo.
    - Estructura:
      - Objetivo: registrar una licencia nueva y asociarla al cliente correcto.
      - Requisitos previos: usuario con permisos y cliente ya creado.
      - Pasos: entrar en el menú de licencias, pulsar **Crear**, rellenar cliente, tipo de licencia, fechas de inicio y fin y, si aplica, número de serie/clave, guardar y revisar que aparece bien en el listado.
      - Resumen rápido: tres líneas con el menú y los campos mínimos que no se pueden olvidar.
  2. Guía “Cómo imputar horas a un proyecto” ( `docs/vrcardio/imputar_horas.md` )
    - Orientada a técnicos que necesitan registrar el tiempo dedicado a proyectos de VRCardio.
    - Estructura:
      - Objetivo: dejar claro cómo imputar horas en tareas de proyecto para que queden en los partes de horas.
      - Requisitos previos: acceso a Proyectos/Partes de horas y proyecto/tareas ya creados.
      - Pasos: abrir el proyecto y la tarea en Odoo, ir a la pestaña de partes de horas, añadir una línea indicando usuario, descripción breve y número de horas (por ejemplo 1.5 para 1h 30min) y guardar.
      - Resumen rápido: recordar el menú exacto y que siempre hay que guardar la tarea y comprobar que las horas se ven en el histórico.
-



## Publicación en el Docusaurus del Proyecto 1

---

El Proyecto 1 ya estaba configurado con un sidebar autogenerado que construye el menú a partir de la estructura de la carpeta `docs` usando `dirName: '.'`, por lo que no ha sido necesario modificar el script de `sidebars.js`.

Al crear la carpeta `docs/vrcardio` y los dos ficheros Markdown:

- Docusaurus ha detectado automáticamente las nuevas páginas al reiniciar `npm run start`.
- En `http://localhost:3000/docs` aparece una nueva sección relacionada con VRCardio en el menú lateral, y dentro de ella las dos guías rápidas creadas esta semana.
- Con esto se cumple el requisito de que la documentación de los procedimientos resida en el Docusaurus del Proyecto 1 y quede accesible para el staff de VRCardio desde el mismo portal que el resto de documentación del proyecto.

---

## Conclusiones

Durante la semana 11 se ha conseguido:

1. Preparar el entorno de documentación en la instancia de AWS, clonando el repositorio del Proyecto 1 y dejando Docusaurus funcionando en modo desarrollo tras ejecutar `npm install` y `npm run start`.
2. Crear dos guías rápidas específicas para VRCardio (`licencias_alta.md` e `imputar_horas.md`) dentro de `docs/vrcardio`, orientadas a personal de administración y técnicos para documentar cómo dar de alta licencias y cómo imputar horas en proyectos.
3. Integrar estas nuevas guías en el Docusaurus del Proyecto 1 sin tocar la configuración del sidebar, aprovechando el sistema autogenerado para que aparezcan automáticamente en el menú de documentación y queden accesibles junto al resto de contenidos.

---

## Semana 12 – Arquitectura final y credenciales

---

### Introducción

---

**Objetivo de la semana:** Documentar de forma visual y resumida la arquitectura final del ERP basado en Odoo Community 16 (AWS + Docker + Nginx + PostgreSQL), y dejar recogido un cuadro claro con los tipos de credenciales que se usan en el sistema.

---

# Mapa de arquitectura

Usuarios → Internet → erp.spikatech.com (DNS)

```
→ Nginx (HTTPS, proxy inverso)
→ Contenedor Odoo (odoo-prod-app, puerto 8069)
→ Contenedor PostgreSQL (odoo-prod-db, puerto 5432)
→ Volúmenes de datos + Backups diarios + NAS (planificado)
```

- **Usuarios**

- Acceden desde el navegador a: <https://erp.spikatech.com>.

- **Instancia AWS**

- Servidor Amazon Linux con Docker y Nginx.
- Puertos abiertos: 22 (SSH), 80 (HTTP), 443 (HTTPS).
- Puertos internos solo locales: 8069 (Odoo), 5432 (PostgreSQL).

- **Nginx**

- Recibe todo el tráfico a erp.spikatech.com.
- Redirige HTTP → HTTPS.
- Usa certificados Let's Encrypt.
- Reenvía las peticiones a Odoo en 127.0.0.1:8069.

- **Docker**

- Contenedor odoo-prod-app (Odoo 16) conectado a:
- Contenedor odoo-prod-db (PostgreSQL 15) con usuario odooprod.
- Volúmenes:
  - Configuración y addons de Odoo.
  - Datos de Odoo.
  - Datos de la base de datos.

- **Backups**

- Script diario que hace *pgdump* de la base de datos y guarda ficheros *urcardioYYYYMMDDHHMM.sql*.
- Copias previstas a NAS Synology mediante *rsync* (diseñado, pendiente de permisos).

## Cuadro de credenciales

*Nota: aquí se documenta qué credenciales existen y para qué sirven, pero las contraseñas reales se gestionan mediante variables de entorno en el servidor, no expuestas en el código ni en la documentación.*

#### [Instancia AWS EC2]

- Usuario sistema: ec2-user
- Clave SSH: [CLAVEPRIVADAARCHIVO\_PROTEGIDO]
- Host: ec2-56-228-82-108.eu-north-1.compute.amazonaws.com

#### [Base de datos PostgreSQL (contenedor odoo-prod-db)]

- Usuario BD Odoo: odoo prod
- Contraseña BD Odoo: [CONTRASEÑA\_OCULTA]
- Base de datos usada por Odoo: postgres (donde está la BD vrcardio)
- Definido en: docker-compose.yml (POSTGRESUSER, POSTGRESPASSWORD)
- Referenciado en: odoo.conf (dbuser=odoo prod, dbpassword=[CONTRASEÑA\_OCULTA])
- Puerto: 5432 (solo red interna Docker)

#### [Odoo – Usuarios de aplicación]

- Administrador ERP:
  - Email (login): admin.its@spikatech.com
  - Password (asistente de creación): [CONTRASEÑA\_OCULTA]
  - Rol: configuración global, instalación de apps, gestión de usuarios
- Usuario API (servicios externos):
  - Email (login): api.user@spikatech.com
  - Password: [CONTRASEÑA\_OCULTA]
  - Uso: scripts Python vía XML-RPC (odooapitest.py, importodoodata.py)
  - Base de datos objetivo: vrcardio
  - Endpoints usados:
    - <https://erp.spikatech.com/xmlrpc/2/common>
    - <https://erp.spikatech.com/xmlrpc/2/object>

#### [Nginx / HTTPS]

- Dominio ERP: erp.spikatech.com
- Certificados Let's Encrypt:
  - Ruta: /etc/letsencrypt/live/erp.spikatech.com/
  - Archivos: fullchain.pem, privkey.pem
- Comando de emisión:
  - sudo certbot --nginx -d erp.spikatech.com

#### [Backups]

- Script: /home/ec2-user/proyecto-erp/scripts/backup\_vrcardio.sh
- Carpeta de copias: /home/ec2-user/proyecto-erp/backups
- Nombre de ficheros: vrcardio\_YYYYMMDDHHMM.sql
- Tarea cron:

- `03 ** */home/ec2-user/proyecto-erp/scripts/backup_vrcardio.sh >> /home/ec2-user/backup.log 2>&1`

[NAS Synology]

- Usuario remoto: odoo
  - Host: dreamtechnologycl.fr3.quickconnect.to
  - Carpeta remota: /odoo/
  - Sincronización prevista:
    - `rsync -avz /home/ec2-user/proyecto-erp/backups/ \`  
`odoo@dreamtechnologycl.fr3.quickconnect.to:/odoo/`
  - Estado: diseño terminado, pendiente de resolver "Permission denied (password)".
- 

## Conclusiones

---

Durante la semana 12 se ha conseguido:

1. Dejar documentado de forma clara y esquemática el mapa de arquitectura final del ERP: instancia AWS con Docker, contenedores de Odoo y PostgreSQL, Nginx como proxy inverso y sistema de copias de seguridad diarias.
  2. Elaborar un cuadro de credenciales que identifica los tipos de usuarios y accesos (sistema, base de datos, Odoo, API, HTTPS y NAS), facilitando la gestión segura de contraseñas y el traspaso de conocimiento a futuros administradores del sistema.
- 

## Conclusión

---

Al terminar el proyecto, el resultado es un Odoo 16 en producción en AWS, accesible por <https://erp.spikatech.com>, con los módulos necesarios configurados, los datos reales de Spika Tech cargados en la compañía y los flujos principales listos: venta de licencias, seguimiento comercial, proyectos de implantación con imputación de horas y gestión básica de empleados.

En la parte técnica he pasado por el despliegue con Docker, la configuración de Nginx y HTTPS, la automatización de backups y el uso de la API de Odoo desde scripts de Python para consultar e importar datos. En la parte funcional he configurado Odoo para que reúna en un solo sitio clientes, licencias, proyectos, ventas y empleados, de forma que podría utilizarse como ERP real en VRCardio / Spika Tech y ampliarse con más usuarios, módulos e integraciones en siguientes fases.